

```
"""
```

```
Crypto + Forex News Bot for Telegram
```

```
-----
```

```
Polls free RSS feeds for crypto and forex news and posts new items  
to a Telegram chat/channel on a schedule.
```

SETUP

1. Create a bot: message @BotFather on Telegram -> /newbot -> copy the token
2. Get your chat ID:
 - For a personal chat: message your new bot once, then visit
<https://api.telegram.org/bot<TOKEN>/getUpdates> and read the "chat":{"id": ..
 - For a channel: add the bot as admin, then use the channel's @username
(e.g. "@my_news_channel") as TELEGRAM_CHAT_ID
3. Set the environment variables below (or edit the constants directly).
4. pip install -r requirements.txt
5. python bot.py

DEPLOYMENT

```
Runs forever in a loop. Deploy on any always-on host: a VPS, Raspberry Pi,  
Railway, Render (background worker), Fly.io, or a systemd service.
```

```
"""
```

```
import os
import time
import logging
import sqlite3
from datetime import datetime, timezone

import feedparser
import requests

# -----
# CONFIG
# -----

TELEGRAM_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN", "PUT_YOUR_TOKEN_HERE")
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID", "PUT_YOUR_CHAT_ID_HERE")

# How often to check feeds, in seconds (default: every 10 minutes)
POLL_INTERVAL_SECONDS = int(os.environ.get("POLL_INTERVAL_SECONDS", "600"))

# Free, no-API-key-required RSS feeds
FEEDS = {
    "crypto": [
```

```

        ("CoinDesk", "https://www.coindesk.com/arc/outboundfeeds/rss/"),
        ("CryptoPotato", "https://cryptopotato.com/feed/"),
        ("CryptoPanic", "https://cryptopanic.com/news/rss/"),
        ("Cointelegraph", "https://cointelegraph.com/rss"),
    ],
    "forex": [
        ("ForexLive", "https://www.forexlive.com/feed/news"),
        ("Investing.com Forex", "https://www.investing.com/rss/news_1.rss"),
        ("FXStreet", "https://www.fxstreet.com/rss/news"),
    ],
}

DB_PATH = os.environ.get("DB_PATH", "seen_articles.db")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
)
log = logging.getLogger("news_bot")

# -----
# STORAGE - tracks which articles have already been posted
# -----

def init_db():
    conn = sqlite3.connect(DB_PATH)
    conn.execute(
        """CREATE TABLE IF NOT EXISTS seen (
            link TEXT PRIMARY KEY,
            posted_at TEXT
        )"""
    )
    conn.commit()
    return conn

def already_seen(conn, link: str) -> bool:
    cur = conn.execute("SELECT 1 FROM seen WHERE link = ?", (link,))
    return cur.fetchone() is not None

def mark_seen(conn, link: str):
    conn.execute(
        "INSERT OR IGNORE INTO seen (link, posted_at) VALUES (?, ?)",
        (link, datetime.now(timezone.utc).isoformat()),
    )

```

```

conn.commit()

# -----
# TELEGRAM
# -----

def send_telegram_message(text: str):
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    payload = {
        "chat_id": TELEGRAM_CHAT_ID,
        "text": text,
        "parse_mode": "HTML",
        "disable_web_page_preview": False,
    }
    try:
        resp = requests.post(url, data=payload, timeout=15)
        if resp.status_code != 200:
            log.error("Telegram error %s: %s", resp.status_code, resp.text)
    except requests.RequestException as e:
        log.error("Failed to send Telegram message: %s", e)

def format_message(category: str, source: str, title: str, link: str) -> str:
    emoji = "🟡" if category == "crypto" else "🔴"
    return f"{emoji} <b>[{category.upper()} | {source}]</b>\n{title}\n{link}"

# -----
# FEED POLLING
# -----

def check_feeds(conn):
    for category, sources in FEEDS.items():
        for source_name, url in sources:
            try:
                parsed = feedparser.parse(url)
            except Exception as e:
                log.error("Error parsing feed %s: %s", source_name, e)
                continue

            if parsed.bozo and not parsed.entries:
                log.warning("Feed %s returned no entries (possibly down)", source_name)
                continue

            # Only look at the latest few entries per feed per cycle
            for entry in parsed.entries[:8]:

```

```

        link = entry.get("link")
        title = entry.get("title", "Untitled")
        if not link:
            continue
        if already_seen(conn, link):
            continue

        message = format_message(category, source_name, title, link)
        send_telegram_message(message)
        mark_seen(conn, link)
        log.info("Posted: [%s] %s", source_name, title)

        # avoid hitting Telegram's rate limits when many new items appear
        time.sleep(1.5)

# -----
# MAIN LOOP
# -----

def main():
    if "PUT_YOUR" in TELEGRAM_BOT_TOKEN or "PUT_YOUR" in TELEGRAM_CHAT_ID:
        log.error(
            "Set TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID as environment "
            "variables (or edit the constants in bot.py) before running."
        )
        return

    conn = init_db()
    log.info("Crypto + Forex news bot started. Polling every %ss.", POLL_INTERVAL)

    # On first run, mark everything currently in the feeds as "seen" so you
    # don't get flooded with months of old articles. Comment this out if you
    # want the backlog posted immediately.
    log.info("Priming database with current feed contents (no messages sent)...")
    for category, sources in FEEDS.items():
        for source_name, url in sources:
            try:
                parsed = feedparser.parse(url)
                for entry in parsed.entries[:8]:
                    link = entry.get("link")
                    if link:
                        mark_seen(conn, link)
            except Exception as e:
                log.error("Error priming feed %s: %s", source_name, e)
    log.info("Priming complete. Now watching for new articles.")

```

```
while True:
    try:
        check_feeds(conn)
    except Exception as e:
        log.error("Unexpected error in main loop: %s", e)
        time.sleep(POLL_INTERVAL_SECONDS)

if __name__ == "__main__":
    main()
```